

JavaScript: перебираючи методи масивів

Головна ідея: замість ручних циклів `for` / `for...of`, масиви мають вбудовані методи, які приймають колбек-функцію і самі перебирають елементи. Кожен метод вирішує одне конкретне завдання: трансформація (`map`), відбір (`filter`), пошук (`find`), перевірка умови (`every` / `some`), згортання в одне значення (`reduce`), сортування (`toSorted`). Методи можна поєднувати в ланцюжки.

1. Функції як значення та колбеки

Функцію можна зберегти у змінній або передати як аргумент іншій функції — так само, як число чи рядок.

```
function greet(name) { return `Welcome ${name}!`; }

greet("Mango"); // виклик -- повертає результат
greet;          // без дужок -- це саме посилання на функцію
```

Термін	Значення
Колбек-функція (callback)	функція, передана іншій функції як аргумент, щоб та її викликала
Функція вищого порядку (higher order function)	функція, яка приймає іншу функцію як параметр або повертає функцію
Інлайн-колбек	колбек, оголошений прямо на місці виклику, без окремого імені

```
function registerGuest(name, callback) {
  console.log(`Registering ${name}!`);
  callback(name); // виклик переданої функції
}

registerGuest("Mango", function greet(name) {
  console.log(`Welcome ${name}!`); // інлайн-колбек
});
```

2. Стрілочні функції (arrow functions)

```
// Звичайна функція
function classicAdd(a, b) { return a + b; }

// Стрілочна -- завжди функціональний вираз, присвоюється змінній
const arrowAdd = (a, b) => { return a + b; };

// Неявне повернення (без {} і return) -- якщо тіло це один вираз
const shortAdd = (a, b) => a + b;

// Один параметр -- дужки можна опустити
const double = x => x * 2;

// Без параметрів -- дужки обов'язкові
const greet = () => console.log("Hello!");
```

У стрілочних функцій **немає** власного `arguments` -- для збору аргументів використовуй `...rest`: `const sum = (...args) => {...}`. Саме короткий синтаксис і неявне повернення роблять стрілки зручними для колбеків у перебираючих методах.

3. forEach() -- заміна циклу

```
array.forEach((element, index, array) => {
  // виконується для кожного елемента
});

[5, 10, 15].forEach((num, i) => console.log(`Index ${i}: ${num}`));
```

Властивість	Значення
Повертає	<code>undefined</code> завжди -- не можна ланцюжком викликати інший метод після
Переривання	неможливе -- завжди перебирає весь масив до кінця (на відміну від <code>for</code> з <code>break</code>)
Параметри колбека	<code>element</code> (обов'язково використовується), <code>index</code> , <code>array</code> -- останні необов'язкові

4. Чисті функції (pure functions)

Чиста функція не змінює передані аргументи і завжди повертає той самий результат для тих самих вхідних даних. Більшість перебираючих методів масиву -- чисті: вони створюють **новий** масив, не чіпаючи оригінал.

```
// ✗ Нечиста -- мутує вихідний масив
const dirtyMultiply = (array, value) => {
  for (let i = 0; i < array.length; i++) array[i] *= value;
};

// ✓ Чиста -- повертає новий масив, оригінал незмінний
const pureMultiply = (array, value) => array.map(el => el * value);
```

5. map() і flatMap()

map(callback) -- трансформація

```
const planets = ["Earth", "Mars", "Venus"];
const upper = planets.map(p => p.toUpperCase()); // ["EARTH", "MARS", "VENUS"]
// planets не змінився; новий масив ТІЄЇ Ж довжини
```

```
// Масив об'єктів -- дістати значення однієї властивості
const students = [{ name: "Mango", score: 83 }, { name: "Poly", score: 59 }];
const names = students.map(s => s.name); // ["Mango", "Poly"]
```

flatMap(callback) -- маp + "розгладжування" на 1 рівень

```
const students = [
  { name: "Mango", courses: ["math", "physics"] },
  { name: "Poly", courses: ["science", "math"] },
];

students.map(s => s.courses); // [["math","physics"], ["science","math"]] -- вкладено
students.flatMap(s => s.courses); // ["math","physics","science","math"] -- плаский масив
```

6. filter() і find()

	filter(callback)	find(callback)
Повертає	новий масив усіх елементів, що задовольнили умову	перший елемент, що задовольнив умову (або undefined)
Якщо нічого не знайдено	порожній масив []	undefined
Коли зупиняється	ніколи -- перевіряє всі елементи	одразу після першого збігу

```
const values = [51, -3, 27, -68, 42];
values.filter(v => v >= 0); // [51, 27, 42] -- усі, що підходять

const students = [{ name: "Mango", score: 83 }, { name: "Ajax", score: 37 }];
students.find(s => s.score < 50); // { name: "Ajax", score: 37 } -- перший знайдений
```

findIndex(callback) -- працює як find() , але повертає індекс знайденого елемента (або -1 , якщо не знайдено) замість самого елемента. Зручно, коли потрібно потім видалити/замінити елемент за індексом.

7. every() і some()

	every(callback)	some(callback)
Повертає true , якщо	усі елементи задовольняють умову	хоча б один елемент задовольняє умову
Зупиняється	як тільки колбек поверне false	як тільки колбек поверне true

```
const products = [{ name: "apple", qty: 2 }, { name: "plum", qty: 0 }];
products.every(p => p.qty > 0); // false -- не всі в наявності
products.some(p => p.qty > 0); // true -- хоча б один є
```

8. reduce() -- згортання в одне значення

```
array.reduce((accumulator, element, index, array) => {
  // повертає значення для наступної ітерації
}, initialValue);
```

Параметр	Значення
accumulator	проміжний результат -- те, що повернув колбек на попередній ітерації
initialValue	стартове значення акумулятора (2-й аргумент reduce , не обов'язковий, але рекомендований)

```
const total = [2, 7, 3].reduce((acc, num) => acc + num, 0);
// 0+2=2 -> 2+7=9 -> 9+3=12
console.log(total); // 12

// Масив об'єктів -- сума властивості
const students = [{ score: 83 }, { score: 59 }, { score: 37 }];
const totalScore = students.reduce((acc, s) => acc + s.score, 0); // 179
```

reduce() -- найуніверсальніший метод: може замінити map , filter і навіть їх комбінацію, але через це й найважче читається. Використовуй, коли треба звести масив саме до одного значення (число, об'єкт, рядок).

9. toSorted() -- сортування без мутації

```
const scores = [61, 19, 74, 35];
const sorted = scores.toSorted(); // за замовчуванням -- як рядки!
console.log(scores); // [61, 19, 74, 35] -- оригінал не змінився
console.log(sorted); // [19, 35, 61, 74]
```

За замовчуванням елементи приводяться до рядків перед порівнянням -- тому числа на кшталт [27, 2, 41] без колбека сортуються "як текст" і дають неочікуваний результат. Завжди передавай функцію порівняння для чисел.

Власний порядок сортування (compareFunction)

```
// Числа: за зростанням / спаданням
arr.toSorted((a, b) => a - b); // зростання
arr.toSorted((a, b) => b - a); // спадання

// Рядки: localeCompare()
arr.toSorted((a, b) => a.localeCompare(b)); // алфавіт
arr.toSorted((a, b) => b.localeCompare(a)); // у зворотному порядку

// Масив об'єктів -- за значенням властивості
students.toSorted((a, b) => a.score - b.score); // за зростанням балів
students.toSorted((a, b) => a.name.localeCompare(b.name)); // за іменем
```

sort() vs **toSorted()**: старий метод `array.sort()` робить те саме, але **мутує оригінальний масив** (змінює його на місці). `toSorted()` -- сучасний (ES2023), незмінний аналог: завжди повертає новий масив, оригінал лишається недоторканим. У новому коді варто надавати перевагу `toSorted()` .

10. Ланцюжки методів (method chaining)

Кожен наступний метод викликається на результаті попереднього -- без проміжних змінних.

```
// Без ланцюжка -- зайва проміжна змінна
const sortedByScore = students.toSorted((a, b) => a.score - b.score);
const names = sortedByScore.map(s => s.name);

// Той самий результат ланцюжком
const names = students
  .toSorted((a, b) => a.score - b.score)
  .map(s => s.name);

// Довший приклад: унікальні предмети, відсортовані за алфавітом
const uniqueSortedCourses = students
  .flatMap(s => s.courses)
  .filter((course, i, arr) => arr.indexOf(course) === i) // залишити тільки унікальні
  .toSorted((a, b) => a.localeCompare(b));
```

Ланцюжок зазвичай тримають у межах 2-3 методів: кожен виклик -- це ще одне повне перебирання масиву, довгі ланцюжки шкодять і читабельності, і продуктивності.

11. Швидкий довідник

Метод	Повертає	Коли використовувати
<code>forEach(cb)</code>	<code>undefined</code>	виконати дію для кожного елемента (лог, побічний ефект)
<code>map(cb)</code>	новий масив тієї ж довжини	трансформувати кожен елемент
<code>flatMap(cb)</code>	новий "розгладжений" масив	як <code>map()</code> , коли колбек повертає масиви
<code>filter(cb)</code>	новий масив (коротший або той самий)	відібрати елементи за умовою
<code>find(cb)</code>	перший елемент або <code>undefined</code>	знайти один елемент за умовою
<code>findIndex(cb)</code>	індекс або <code>-1</code>	знайти індекс елемента за умовою
<code>every(cb)</code>	<code>true</code> / <code>false</code>	чи задовольняють умову УСІ елементи
<code>some(cb)</code>	<code>true</code> / <code>false</code>	чи задовольняє умову ХОЧА Б ОДИН елемент
<code>reduce(cb, init)</code>	будь-яке значення	звести масив до одного результату (сума, об'єкт тощо)
<code>toSorted(cb)</code>	новий відсортований масив	сортування без зміни оригіналу (сучасний варіант)
<code>sort(cb)</code>	той самий масив (мутований)	старий варіант сортування -- змінює оригінал
<code>a.localeCompare(b)</code>	число (<0 / 0 / >0)	компаратор для сортування рядків